

IAW · UNIDAD 6

La despensa y el libro de recetas

Los datos del Club, en MariaDB · la configuración del servidor, en Git

RECORDAMOS

La cocina necesita una despensa



Camarero

Nginx

Cocina

PHP y Python

Despensa

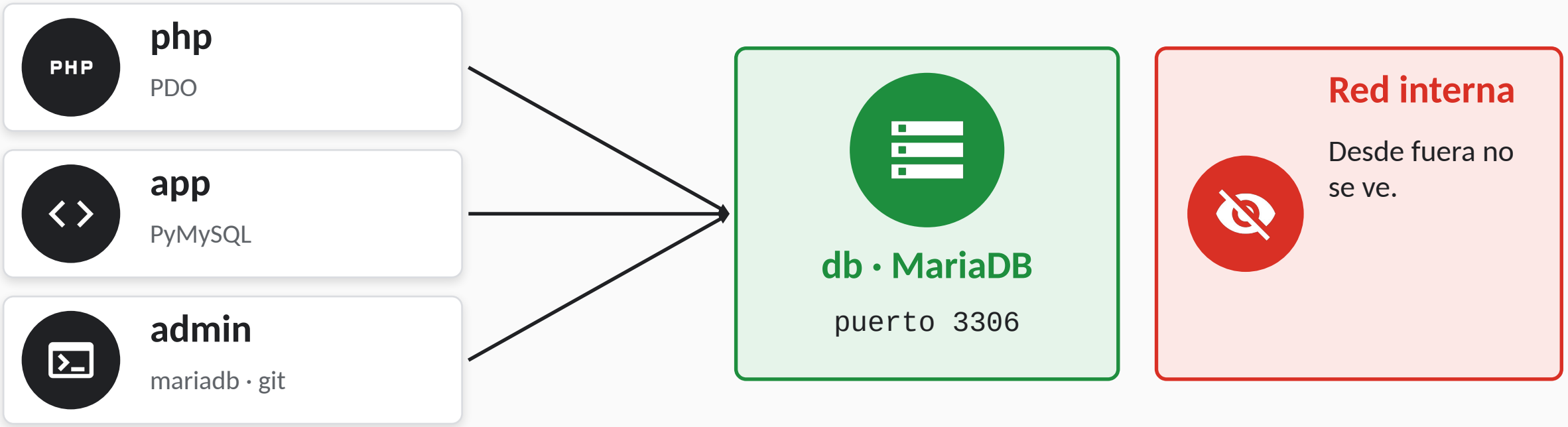
MariaDB (hoy)

Hasta ahora, los datos estaban dentro del programa: se pierden al reiniciar, cada worker tiene su copia y PHP y Python no los comparten.

HOY — La despensa (MariaDB) y quién puede entrar en ella.

QUÉ ES MARIADB

Un servidor aparte, solo para los datos



La base de datos nunca se abre a internet

MARIADB = MYSQL — La continuación libre de MySQL: se instala, se usa y se conecta igual.

ENTRAR

El cliente mariadb, como root, dentro del servidor

terminal

```
$ docker compose exec db mariadb -u root -p
Enter password: ••••••••

SHOW DATABASES;
USE club;
SHOW TABLES;
SELECT id, usuario, email FROM socios;
```

salida real

```
+----+---------+----------------------+
| id | usuario | email                |
+----+---------+----------------------+
|  1 | ana     | ana@club.iaw.local   |
|  2 | luis    | luis@club.iaw.local  |
|  3 | marta   | marta@club.iaw.local |
+----+---------+----------------------+
```

information_schema · mysql · performance_schema · sys: son de MariaDB. No se tocan.

ACUÉRDATE — Cada orden SQL termina en ; · para salir: EXIT;

SQL JUSTO

Crear, añadir, leer… y el UPDATE sin WHERE

SQL (como root)

```
CREATE TABLE libros (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(150) NOT NULL,
  autor VARCHAR(100) NOT NULL,
  anio SMALLINT NOT NULL, paginas SMALLINT NOT NULL);
INSERT INTO libros (titulo, autor, anio, paginas)
  VALUES ('Nada', 'Carmen Laforet', 1945, 320), …;
SELECT titulo, anio FROM libros ORDER BY anio;
```

UPDATE sin WHERE

UPDATE libros SET titulo='???';
→ cambia LAS CUATRO filas.
No hay papelera ni deshacer.

UPDATE y DELETE, siempre con WHERE

LO QUE TE SALVA — Una copia de seguridad (sesión 3).

Cada uno, lo justo

SQL (como root)

```
CREATE USER 'club_app'@'%' IDENTIFIED BY 'una-contraseña-larga';
GRANT SELECT, INSERT, UPDATE ON club.* TO 'club_app'@'%';
SHOW GRANTS FOR 'club_app'@'%';
```

`'club_app'@'%'`

nombre + desde dónde

`SELECT, INSERT, UPDATE`

leer, añadir, cambiar. Sin DELETE ni DROP

`ON club.*`

solo en su base (nunca *.*)

La aplicación nunca entra como root

MÍNIMO PRIVILEGIO — Un usuario por aplicación, con permiso solo en su base y solo para lo que necesita.

DEMO

club_app no puede borrar · root, solo en casa

en admin: docker compose exec admin bash

```
$ mariadb -h db -u club_app -p club -e "DROP TABLE libros"
ERROR 1142 (42000) at line 1: DROP command denied to user 'club_app'@'192.168.32.4' for table `club`.`libros`
```

como root, dentro del servidor

```
SELECT User, Host FROM mysql.user;
-- root | %   ← desde cualquier sitio
DROP USER 'root'@'%';
```

Y la web se cae

libros.php: 1045 Access denied for user 'root'.
Usaba root: se arregla en la sesión 2.

root sigue existiendo, pero solo como root@localhost

LAS LÍNEAS ------ — El cliente de admin repite la orden entre guiones antes del error. Es normal.

TU TURNO

La despensa del Club → 4 de 10

1	Entra como root	`docker compose exec db mariadb -u root -p`
2	La tabla libros	`CREATE TABLE + INSERT de las 4 lecturas`
3	El usuario de la app	`CREATE USER club_app + GRANT SELECT, INSERT, UPDATE`
4	Pruébalo desde admin	`mariadb -h db -u club_app -p → DROP: 1142`
5	Root, solo en casa	`DROP USER 'root'@'%'`
6	Comprueba	`bash comprobar.sh → 4 de 10`

APUNTA LA CONTRASEÑA — La de club_app la necesitas el miércoles.

HOY

La cocina pide a la despensa

1

Conectar PHP y Python

PDO y PyMySQL: dónde, qué base, quién y con qué contraseña

2

Los secretos, fuera del código

La contraseña en club.env, no en db.php

3

La inyección SQL

El ataque más famoso de la web, y cómo se evita

El DSN dice a qué despensa ir

www/club/inc/db.php

```php
$dsn = 'mysql:host=db;dbname=club;charset=utf8mb4';
$pdo = new PDO($dsn, $usuario, $clave, [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
]);
$libros = $pdo->query('SELECT titulo, autor, anio FROM libros')->fetchAll();
```

`mysql:`

el tipo (MariaDB = MySQL)

`host=db`

dónde está

`dbname=club`

qué base

`charset=utf8mb4`

las tildes y los emojis

EN DEBIAN — sudo apt install php-mysql (en el laboratorio ya está).

Las mismas cuatro preguntas

app/app.py

```
c = pymysql.connect(
    host=os.environ["DB_HOST"],
    database=os.environ["DB_NAME"],
    user=os.environ["DB_USER"],
    password=os.environ["DB_PASS"],
    charset="utf8mb4")
```

¿Dónde?	`host`
¿Qué base?	`database / dbname`
¿Quién?	`user`
¿Con qué contraseña?	`password`

https://localhost:8449/libros.php

```
Lecturas del trimestre (4) … Conectado a la base de datos como: club_app@%
```

LA API — Ya está escrita: solo le falta la contraseña (y restart app, como en la UD5).

LOS SECRETOS, FUERA DEL CÓDIGO

La contraseña, en club.env

Antes: en db.php

new PDO($dsn, 'root', 'raiz-del-lab')
escrito en el código… que acaba en Git.

servidor/club.env

```
DB_HOST=db
DB_NAME=club
DB_USER=club_app
DB_PASS=…
```

www/club/inc/db.php (después)

```php
$cfg = leer_env('/etc/club/club.env');
return new PDO($dsn, $cfg['DB_USER'], $cfg['DB_PASS'], [ … ]);
```

Fuera de la web · fuera de Git · solo lo lee el servicio. PHP lo lee en cada petición; Python, al arrancar.

SI CAMBIAS club.env — PHP: al recargar · la API: docker compose restart app.

DEMO · INYECCIÓN SQL

Cuando el dato se convierte en orden

www/club/buscar.php (así viene)

```
$sql = "SELECT titulo, autor, anio FROM libros WHERE titulo LIKE '%$q%'";
```

lo que escribe el atacante en el buscador

```
x' UNION SELECT usuario, clave, email FROM socios -- -
```

salida real del buscador

```
ana — $2y$10$Hc8vZbXq4m1Yw0tPpQ7NeuXr8m6sJk2bQ9dW… (ana@club.iaw.local)
luis — $2y$10$Rk2pL9sD4fG6hJ8kL0zX1cV3bN5mQ7wE… (luis@club.iaw.local)
```

Los socios, en un buscador de libros

CON club_app TAMBIÉN — Tiene SELECT en club.*: el mínimo privilegio limita el daño, no evita esto.

CONSULTAS PREPARADAS

El SQL con un hueco; el dato, aparte

www/club/buscar.php (bien)

```php
$st = $pdo->prepare('SELECT titulo, autor, anio FROM libros WHERE titulo LIKE ?');
$st->execute(['%' . $q . '%']);
$resultado = $st->fetchAll();
```

app/app.py (Python: %s y el dato aparte)

```python
cur.execute("SELECT … FROM libros WHERE titulo LIKE %s", ("%" + q + "%",))
```

El mismo ataque

0 resultados. Y ?q=Nada sigue encontrando «Nada».

No vale

Quitar comillas a mano. Ni htmlspecialchars (eso es XSS).

LA REGLA — Nunca pegues un dato del usuario dentro de un SQL.

CUANDO LA CONEXIÓN FALLA

Cuatro números que te dicen dónde mirar

2002	**No llega**	`getaddrinfo for dbx failed · host mal o MariaDB parado`
1045	**Usuario o contraseña**	`(using password: NO) = no ha mandado contraseña`
1044	**Sin permiso en esa base**	`Access denied … to database 'biblioteca'`
1142	**Esa orden, no**	`DROP command denied to user 'club_app'`

Y EL ERROR EN PANTALLA — Sale entero porque display_errors está encendido: en producción, apagado (UD4).

TU TURNO

La cocina pide a la despensa → 8 de 10

1	La contraseña	`servidor/club.env: DB_PASS=…`
2	La web, con club_app	`db.php: $cfg = leer_env(…) y sin claves escritas`
3	Sin inyección	`buscar.php: prepare(… LIKE ?) + execute([…])`
4	La API	`docker compose restart app → /api/libros`
5	Comprueba	`bash comprobar.sh → 8 de 10`

SI ACABAS ANTES — Rompe club.env a propósito (host, base, contraseña) y lee cada error.

HOY

Copias y Git

1

Copias de la base de datos

mysqldump y, sobre todo, restaurar

2

Un script de copia

Sin contraseñas dentro, con fecha, cada noche

3

La configuración, en Git

Ver qué cambió y volver atrás

COPIAR Y RESTAURAR

mysqldump escribe la receta para volver a crear la base

en admin

```
$ mysqldump -h db -u club_app -p --single-transaction --no-tablespaces club > /copias/club.sql
```

/copias/club.sql (es texto)

```
-- MariaDB dump 10.19-11.8.6-MariaDB
-- Host: db    Database: club
CREATE TABLE `libros` (
…
INSERT INTO `libros` VALUES
(1,'El camino','Miguel Delibes',1950,224),
```

restaurar (root, dentro del servidor)

```
$ cat club.sql | \
  docker compose exec -T db \
  mariadb -u root -p'…' club

-- los cuatro títulos vuelven
```

Una copia que no has restaurado nunca no es una copia

PRUÉBALA — Restaura en otra base (club_prueba) y mira que están las tablas y los datos.

UN SCRIPT DE COPIA

Con fecha, sin contraseñas dentro, cada noche

servidor/copia.sh

```
#!/bin/sh
set -e
. /srv/servidor/club.env        # la contraseña, de aquí
FECHA=$(date +%Y%m%d-%H%M%S)
mysqldump -h "$DB_HOST" -u "$DB_USER" -p"$DB_PASS" \
  --single-transaction --no-tablespaces "$DB_NAME" \
  > "/copias/club-$FECHA.sql"
find /copias -name 'club-*.sql' -mtime +7 -delete
```

cron

30 3 * * *
/srv/servidor/copia.sh

Fuera del servidor

Si se rompe el disco… (en SAD)

```
docker compose exec admin sh copia.sh
Copia hecha: /copias/club-20261030-191117.sql
```

EL SCRIPT VA A GIT — Por eso no lleva la contraseña: la lee de club.env.

EL LIBRO DE RECETAS

La configuración del servidor, en Git

en admin: /srv/servidor

```
$ git init
$ git config user.name "Ana Admin"
$ git config user.email ana@club.iaw.local
$ git status --short
?? club.env          ← ¡la contraseña!
?? copia.sh
?? nginx/
```

Quién lo cambió

Cuándo fecha y hora

Por qué el mensaje del commit

Volver atrás si se rompe

Antes del primer git add: el .gitignore

LO MISMO QUE EN DAW — Pero para /etc/nginx, los scripts y la configuración del servidor.



.GITIGNORE

Los secretos, fuera del repositorio

servidor/.gitignore

```
club.env
*.key
```

club.env



Se queda en el servidor.

en admin

```
$ git add -A
$ git commit -m "Configuración del servidor del Club"
$ git ls-files
.gitignore
copia.sh
nginx/club.conf
$ git check-ignore -v club.env
.gitignore:1:club.env    club.env
```



Si una contraseña llega a Git...

ya está en el historial: borrarla no basta. Hay que CAMBIARLA.



PRIMERO EL .gitignore — Y después el primer git add.

VOLVER ATRÁS

Qué cambió, y cómo se deshace

git diff

```
     location ~ \.php$ {
         try_files $uri =404;
-        fastcgi_pass php:9000;
+        fastcgi_pass php:9000
         include fastcgi_params;
```

nginx -t

```
[emerg] invalid number of
arguments in "fastcgi_pass"
directive in …club.conf:30
test failed
```

Sin commit

`git restore nginx/club.conf`

El historial

`git log --oneline`

Con commit

`git revert HEAD` (un commit que lo deshace)

git revert NO BORRA — Añade un commit que deshace el anterior: queda constancia de lo que pasó.

LABORATORIO GUIADO

Copias y Git → 10 de 10

1	La copia	`copia.sh → docker compose exec admin sh copia.sh`
2	El desastre	`UPDATE libros SET titulo='???'; → restaurar`
3	Los secretos fuera	`servidor/.gitignore: club.env, *.key`
4	El primer commit	`git init · git add -A · git commit -m …`
5	Rómpelo y arréglalo	`club.conf sin ; → git diff → git restore`
6	Comprueba	`bash comprobar.sh → 10 de 10`

SI ACABAS ANTES — Haz un commit con un cambio malo y deshazlo con git revert HEAD.

LO QUE NOS LLEVAMOS DE LA UD6

Resumen de puntos clave

- MariaDB: un servidor aparte (3306), solo en la red interna
- Un usuario por app: GRANT SELECT, INSERT, UPDATE ON club.*
- Las contraseñas en club.env: fuera de la web y de Git
- Consultas preparadas: el dato, aparte del SQL
- mysqldump + restaurar (si no la has restaurado, no es copia)
- Git: .gitignore primero · diff · restore · revert

LO SIGUIENTE

Práctica 2 y UD7 · WordPress

- Bloque C de Radio Marea: su base y su Git
- WordPress = Nginx + PHP + MariaDB
- Todo lo de este trimestre, junto

La cocina saca de la despensa solo lo que puede; y el libro de recetas guarda cada versión.